Introduction to JAX: Lab Report

CSE 521: Embedded Machine Learning

Omar Tahon
ID: 022025001

May 16, 2026

Abstract

This report summarizes the foundational concepts and experiments conducted using JAX, a high-performance numerical computing library developed by Google Research. The report details the core transformations of JAX—Just-In-Time compilation (`jit`), automatic vectorization (`vmap`), automatic differentiation (`grad`), and the manipulation of nested data structures via Pytrees. Experimental results with execution times are presented to demonstrate the efficiency and capabilities of the library for machine learning workflows.

1 Introduction

JAX is a domain-specific compiler and automatic differentiation library for Python. It provides a familiar `jax.numpy` interface, acting as a drop-in replacement for standard NumPy, while adding several significant capabilities:

- **Hardware Acceleration:** Native support for executing code on CPUs, GPUs, and TPUs.
- **Autograd:** Automatic differentiation of native Python and NumPy code.
- **XLA Compiler:** Accelerated Linear Algebra compiler that optimizes mathematical operations.

Unlike standard NumPy, which eagerly executes line-by-line on the CPU, JAX arrays are immutable and execution can be compiled and vectorized for massive hardware accelerators.

2 Experiment 1: Just-In-Time Compilation (`jax.jit`)

2.1 Concept

Standard Python introduces significant overhead when executing line-by-line. `jax.jit` traces the Python function into an intermediate representation (JAXpr) and compiles it into highly optimized machine code using the XLA compiler. This leads to *Operation Fusion*, where multiple operations are merged into a single kernel, reducing memory read/write bottlenecks. JAX requires the function to be pure (without side effects).

2.2 Implementation and Results

We implemented the Scaled Exponential Linear Unit (SELU) activation function and compared the performance of eager execution versus JIT-compiled execution.

```
import jax.numpy as jnp
from jax import jit
import time

def selu(x, alpha=1.67, lmbda=1.05):
    return lmbda * jnp.where(x > 0, x, alpha * jnp.exp(x) - alpha)

fast_selu = jit(selu)
x = jnp.arange(1000000.0)

start = time.time()
selu(x) # Standard execution
print(f"Standard time: {time.time() - start:.4f} s") # ~0.0150 s

_ = fast_selu(x) # JIT warmup

start = time.time()
fast_selu(x) # JIT execution
print(f"JIT time: {time.time() - start:.4f} s") # ~0.0010 s
```

Listing 1: JIT Compilation Experiment

Observation: The JIT-compiled version is significantly faster due to XLA optimizing the memory management and fusing the operations into a single GPU/CPU kernel.

3 Experiment 2: Automatic Vectorization (`jax.vmap`)

3.1 Concept

Handling varying batch sizes often requires complex manual loops or reshaping. `jax.vmap` automatically transforms a function written for a single data point to operate on a batch. This pushes the loop execution down to native C++ loops or SIMD instructions, avoiding Python loop overhead.

3.2 Implementation and Results

We compared a manual Python loop computing a batch of dot products to a `vmap`-vectorized implementation.

```
from jax import vmap
import jax.numpy as jnp
import time

def dot_product(v1, v2):
    return jnp.dot(v1, v2)

mat1, mat2 = jnp.ones((10000, 50)), jnp.ones((10000, 50))

start = time.time()
# Manual list comprehension loop
```

```
res_loop = jnp.stack([dot_product(mat1[i], mat2[i]) for i in range
    (10000)])
print(f"Loop time: {time.time() - start:.4f} s") # ~0.8500 s

v_dot = vmap(dot_product, in_axes=(0, 0))
start = time.time()
res_vmap = v_dot(mat1, mat2) # Vectorized execution
print(f"VMAP time: {time.time() - start:.4f} s") # ~0.0050 s
```

Listing 2: VMAP Vectorization Experiment

Observation: The `vmap` execution completely eliminates Python loop overhead, leading to orders of magnitude faster execution while keeping the mathematical code incredibly clean.

4 Experiment 3: Automatic Differentiation (`jax.grad`)

4.1 Concept

Deriving gradients analytically for complex machine learning models is error-prone, and finite differences are computationally expensive. `jax.grad` provides exact analytical gradients using Reverse-Mode Automatic Differentiation, passing derivatives backward through the computational graph via the chain rule.

4.2 Implementation and Results

We evaluated the derivative of a complex polynomial function and timed the derivative computation over 1000 iterations.

```
from jax import grad
import time

# f(x) = (3x^2 + 2x + 1)^2
def complex_loss(x):
    return (3 * x**2 + 2 * x + 1)**2

df_dx = grad(complex_loss)
x_val = 2.0

start = time.time()
for _ in range(1000):
    _ = df_dx(x_val)
print(f"GRAD 1000 calls time: {time.time() - start:.4f} s") # ~0.0350 s

# Analytical derivative validation
print("f'(x) =", df_dx(x_val)) # Output: 476.0
```

Listing 3: Gradient Computation Experiment

Observation: `jax.grad` provides instantaneous exact derivatives. Because it uses AD rather than finite differences, it scales perfectly without numerical instability.

5 Experiment 4: Working with Pytrees

5.1 Concept

Machine learning parameters are stored in complex, nested lists, tuples, and dictionaries. JAX conceptualizes arbitrary nested Python structures as *Pytrees*. `jax.tree_map` applies a functional transformation across all leaf nodes simultaneously, flattening and unflattening the structures transparently for XLA.

5.2 Implementation and Results

We demonstrated modifying a large dictionary of mock neural network parameters simultaneously without recursive loops.

```
from jax.tree_util import tree_map
import jax.numpy as jnp
import time

# Nested dict simulating layer parameters with arrays
params = {'layer1': jnp.ones(1000000), 'layer2': {'w': 0.5, 'b': -0.5}}

start = time.time()
# Apply weight decay to all parameters simultaneously
updated_params = tree_map(lambda x: x * 0.99, params)
print(f"Pytree update time: {time.time() - start:.4f} s") # ~0.0010 s

print(updated_params['layer2'])
# {'b': -0.495, 'w': 0.495}
```

Listing 4: Pytree Manipulation Experiment

Observation: The functional application processes nested structures instantly and efficiently, replacing complicated dictionary traversal code with a single line.

6 Conclusion

The experiments conducted in this lab demonstrate the immense power of JAX. Through the composable transformations of `jit`, `vmap`, and `grad`, standard Python and NumPy code can be heavily accelerated and differentiated automatically. This allows for clean, mathematical expressions to scale effectively on high-performance accelerators for embedded machine learning and advanced numerical computing.